

RESEARCH go_cp

RESEARCH go_cp

Go control-plane stack for a Mullvad-parity self-hosted VPN. Latest authoritative versions + best practices, as of **2026-06-25** (access date for every source below). Per §11.4.99/§11.4.150: every claim is cited; latest official docs/releases preferred over training memory.

1. Gin — REST surface

Latest: Gin v1.12.0 (2025-02-28). v1.11.0 (2024-09-20) added HTTP/3 (quic-go), runtime-swappable custom JSON codec, and array form-binding; v1.12.0 added `encoding.UnmarshalText` in uri/query binding and Protobuf content negotiation. Source: <https://github.com/gin-gonic/gin/releases>

Best practice:

- Gin is the thin **REST/JSON edge** (auth callbacks, health, admin, webhooks). Keep the typed RPC surface (mobile/desktop clients, server-streaming) on Connect-RPC (§2) — do NOT duplicate business logic across both. Mount Connect handlers and Gin on the **same http.Server** via a top-level mux: Connect handlers are plain `http.Handlers`, so route `/grpc.* / / <package>.<Service>/` to the Connect mux and everything else to Gin. (<https://connectrpc.com/docs/go/getting-started/>)
 - Use `gin.New()` + explicit middleware (Recovery, structured logger, request-ID), NOT `gin.Default()` in production. Bind+validate with `ShouldBindJSON` and the built-in binding (go-playground/validator) tags.
 - Gin's own perf is rarely the bottleneck for a control plane; the DB round-trips (§3) and Redis (§5) dominate. (<https://gin-gonic.com/>)
-

2. Connect-RPC (connectrpc/connect-go) + buf — typed RPC / server-streaming

Latest: connectrpc.com/connect v1.20.0 (bumps minimum Go to 1.25; v1.19.0 moved to Go 1.24 + Edition 2024 support; supports the **two most recent Go majors** only). Stable, semver, no breaking changes in the 1.x line. Sources: <https://github.com/connectrpc/connect-go/releases> ; <https://pkg.go.dev/connectrpc.com/connect>

Why Connect for a VPN control plane:

- **One handler speaks three protocols simultaneously** — gRPC, gRPC-Web, and the Connect protocol — with NO Envoy/translating proxy. Native binary gRPC-Web means browser/admin frontends talk to the server directly. (<https://connectrpc.com/docs/go/grpc-compatibility/> ; <https://connectrpc.com/docs/multi-protocol/> — gRPC-Web without Envoy)
- **Server-streaming** (the relevant mode for VPN: push device/peer/relay-list updates, live tunnel status, account events). All streaming subtypes (client/server/bidi) work across all three protocols. Caveat: in every protocol a streaming response is always **HTTP 200 OK** even on error — errors are encoded in trailers / end-of-body, never the HTTP status; design clients to read the trailer status, not the response code. Bidi streaming needs HTTP/2 end-to-end. (<https://connectrpc.com/docs/go/streaming/>)
- Connect unary calls are also plain POST (and optional GET for idempotent side-effect-free methods) — debuggable with curl, cacheable. (<https://connectrpc.com/docs/protocol/>)

Codegen with **buf** (modern protoc replacement):

- `buf.yaml` (module config) + `buf.gen.yaml` (plugins). Generate with `protoc-gen-go` + `protoc-gen-connect-go`. `buf lint` + `buf breaking` (against a git ref or the BSR) in CI catch wire-incompatible schema drift before it ships. (<https://connectrpc.com/docs/go/getting-started/> ; <https://buf.build/blog/connect-a-better-grpc>)
 - Pair with **protovalidate** (now v1.0, production-stable) via the `connectrpc/validate-go` interceptor for declarative request validation in the proto schema (`buf.validate` field constraints) instead of hand-written Go checks — validation lives next to the contract. (<https://github.com/connectrpc/validate-go> ; <https://github.com/bufbuild/protovalidate-go/releases>)
-

3. sqlc — typed SQL (NOT an ORM)

Latest: sqlc v1.31.1 (2026-04-22). (Stable docs line is 1.31.x; v1.27.0 introduced managed/local DB support for schema verification.) Sources: <https://github.com/sqlc-dev/sqlc/releases> ; <https://docs.sqlc.dev/en/stable/>

Best practice (Postgres + pgx):

- `sqlc.yaml` **version "2"**, `engine: postgresql`, `sql_package: "pgx/v5"`, `separate_schema: (DDL / migration dir)` and `queries: dirs`. SQL is the source of truth — write `-- name: GetDevice :one` annotated queries, sqlc generates typed Go. (<https://docs.sqlc.dev/en/stable/guides/using-go-and-pgx.html>)
- With `pgx/v5` the generated code uses **pgx native types** (e.g. `pgtype.Timestamptz`, `pgtype.UUID`) directly instead of `database/sql` `Null*` wrappers — cleaner null-handling. Type inference: `int`→`int32`, `bigint`→`int64`, `varchar`→`string`, `timestamptz`→`time.Time`; nullable columns become `pointer/pgtype` automatically. (<https://docs.sqlc.dev/en/latest/reference/datatypes.html>)
- Use overrides in `sqlc.yaml` to map custom domains/enums to Go types (e.g. a `uuid` → `github.com/google/uuid.UUID`, or a `tenant` enum to a typed constant). (<https://docs.sqlc.dev/en/latest/howto/overrides.html>)

- sqlc generates Querier interfaces → trivial to mock at the unit-test layer while integration tests hit a real Postgres (§11.4.27 no-fakes-beyond-unit). Run sqlc in CI with the managed-DB feature so a schema/query mismatch fails the build, not prod.

Driver: **jackc/pgx/v5 v5.10.0 (2026-06-03)** — use the pgxpool connection pool; it is the de-facto Postgres driver and what sqlc's recommended path targets. Source: <https://pkg.go.dev/github.com/jackc/pgx/v5>

4. Postgres Row-Level Security — multi-tenant isolation

Pattern (the canonical, audited 2025-2026 best practice for shared-schema multi-tenancy):

1. **App connects as a non-owner, non-superuser role WITHOUT BYPASSRLS.** If the app role ever owns a table, add `ALTER TABLE t FORCE ROW LEVEL SECURITY` (table owners bypass RLS by default). Reserve BYPASSRLS strictly for the **migration role**, never admin/app. Give admins an explicit admin policy, not BYPASSRLS.
2. **Per-transaction tenant context via SET LOCAL** — `SET LOCAL app.current_tenant = '<id>'` at the start of every transaction; the policy reads `current_setting('app.current_tenant', true)` (the `true` = `missing_ok` avoids errors when unset). **SET LOCAL is load-bearing:** plain SET persists for the whole pooled connection's life, so the next request on that pooled conn inherits the previous tenant's context — the #1 way teams silently break isolation. Always SET LOCAL, always inside an explicit transaction (it auto-resets at COMMIT/ROLLBACK).
3. **Write BOTH USING and WITH CHECK** explicitly on each policy. USING alone covers SELECT/UPDATE-visibility but Postgres does NOT auto-apply it as WITH CHECK on INSERT — so a tenant could insert rows for a tenant they can't read. Spell out both.
4. **Index tenant_id as the leading column** of every composite index. Without it RLS is ~2 orders of magnitude slower; with proper composite indexes, RLS shows no measurable degradation vs app-layer filtering even at ~500 concurrent conns. The bottleneck is always missing indexes, never the policy evaluation.

Composition note for a VPN control plane: the tenant here may be the **account** (or organization). Bind the RLS SET LOCAL to the authenticated account claim from the Connect/Gin auth interceptor, inside the same pgx.Tx the sqlc queries run in — wire it as a single `WithTenantTx(ctx, accountID, fn)` helper so no code path can run a query without first setting the context.

Sources:

- <https://queryplane.com/blog/postgres-row-level-security-in-practice/>
- <https://ricofritzsche.me/mastering-postgresql-row-level-security-rls-for-rock-solid-multi-tenancy/>
- <https://aws.amazon.com/blogs/database/multi-tenant-data-isolation-with-postgresql-row-level-security/>
- <https://www.bytebase.com/reference/postgres/how-to/postgres-row-level-security/>

5. Redis Streams — consumer groups + XAUTOCLAIM dead-letter

Use Redis Streams as the durable work/event bus (device-provisioning jobs, relay health fan-out, async account events). Consumer-group + PEL pattern:

- Each worker reads with **XREADGROUP GROUP <grp> <consumer> COUNT n BLOCK <ms> STREAMS <key> >** and XACKs on success. Unacked messages stay in the **Pending Entries List (PEL)** for that consumer.
- Recover dead/slow peers with **XAUTOCLAIM <key> <grp> <consumer> <min-idle-time> <start>** — atomically scans for PEL entries idle $\geq$ min-idle-time (commonly ~60 000 ms) and transfers ownership to the calling healthy consumer. Preferred over manual XPENDING+XCLAIM loops (single command, cursor-paged). Production loop: **first XAUTOCLAIM stalled work, then XREADGROUP > for new work.**
- **Dead-letter the poison messages.** XAUTOCLAIM/XPENDING expose the per-message **delivery count**; when it exceeds a retry cap (e.g. 5), XADD the payload to a separate <key>:dlq stream + XACK it off the main PEL so it stops being redelivered. The DLQ stream is then inspectable/replayable. Track `delivery_count` + first-seen ts in the DLQ entry for forensics.
- **Redis 8.4+ shortcut:** XREADGROUP ... CLAIM <min-idle-time> folds the reclaim step into the read (single-shot reliable consumers), so you can do retry-caps / poison-routing / stuck-detection without the separate XAUTOCLAIM call. Use it if you can pin Redis $\geq$ 8.4; otherwise the XAUTOCLAIM-then-XREADGROUP two-step is the portable form.
- Periodically trim with XADD ... MAXLEN ~ N or XTRIM MINID so the stream doesn't grow unbounded; the DLQ retention should outlive the main stream for audit.

Sources:

- <https://redis.io/tutorials/redis-backed-job-queue-for-background-workers/>
- <https://redis.io/docs/latest/commands/xclaim/>
- <https://oneuptime.com/blog/post/2026-01-21-redis-dead-letter-queue/view>
- <https://oneuptime.com/blog/post/2026-03-31-redis-handle-consumer-failures-streams/view>
- <https://redis.io/blog/single-shot-reliable-consumers-with-xreadgroup-claim-in-redis-84/> (Redis 8.4 CLAIM)

Go client: use `redis/go-redis/v9` (has XAutoClaim, XReadGroup, XAck, XAdd with MAXLEN). (<https://redis.io/docs/latest/develop/use-cases/streaming/>)

6. Migrations — goose vs Atlas

goose — v3.27.1 (2026-04-24). Lightweight, no DSL: numbered SQL files (--+goose Up / Down) OR Go migration funcs. Tracks state only via a version table; **no drift detection, no auto-planning** — you hand-write every up/down. Best when the

team wants explicit, minimal, reviewable SQL with zero abstraction. Sources: <https://github.com/pressly/goose/releases> ; <https://volomn.com/blog/database-migration-using-atlas-and-goose>

Atlas. Declarative, Terraform-like: you describe the **desired schema** (HCL/SQL), Atlas builds a graph of DB entities, **auto-plans** the migration SQL to reach it, and versions it. Runs migrations in a transaction with an advisory lock (one migration at a time, order/integrity enforced) and **detects drift** between declared schema and live DB. Best when you want declarative schema management + strong CI/CD + drift safety. Sources: <https://atlasgo.io/blog/2022/12/01/picking-database-migration-tool> ; <https://codelit.io/blog/database-migration-tools-comparison>

Recommendation for this stack: the schema is also sqlc's schema: source of truth, and RLS policies (§4) are first-class DDL that **MUST** be reviewed line-by-line — **goose's explicit hand-written SQL is the safer default here** (you want to read the exact `CREATE POLICY ... USING ... WITH CHECK` and `FORCE ROW LEVEL SECURITY` in the diff, not have a planner emit it). Atlas is the upgrade path if/when declarative drift detection across many tables becomes worth the planner abstraction; Atlas can **import an existing goose project** (<https://atlasgo.io/guides/migration-tools/goose-import>), so starting on goose does not lock you out of Atlas later. Either way: migration role runs with `BYPASSRLS`; app role never does (§4).

7. Integration / composition notes

- **Single server, two surfaces:** mount Gin (REST edge) + Connect handlers (typed RPC + server-streaming) on one `http.Server` (HTTP/2 h2c or TLS-ALPN so gRPC
 - bidi streaming work). Auth interceptor (Connect) / middleware (Gin) resolves the account → sets the `RLS SET LOCAL` tenant inside the `pgx.Tx`.
 - **Data path:** Connect/Gin → `WithTenantTx` (sets `app.current_tenant`) → sqlc Querier over `pgxpool` → RLS-enforced Postgres. No query escapes the tenant context.
 - **Async path:** control-plane writes a job/event → Redis Stream → worker pool (`XREADGROUP+XAUTOCLAIM+DLQ`) → effects (relay config push, email, etc.).
 - **Schema lifecycle:** goose migrations are the one DDL source; sqlc reads the same schema/ dir so generated Go always matches the migrated DB; CI runs goose-up against an ephemeral Postgres + sqlc verify so schema/query/code drift fails the build.
-

Sources verified

- <https://github.com/gin-gonic/gin/releases> — accessed 2026-06-25 (Gin v1.12.0 / v1.11.0)
- <https://gin-gonic.com/> — accessed 2026-06-25
- <https://github.com/connectrpc/connect-go/releases> — accessed 2026-06-25 (connect-go v1.20.0 / v1.19.0)
- <https://pkg.go.dev/connectrpc.com/connect> — accessed 2026-06-25
- <https://connectrpc.com/docs/go/grpc-compatibility/> — accessed 2026-06-25
- <https://connectrpc.com/docs/go/streaming/> — accessed 2026-06-25
- <https://connectrpc.com/docs/go/getting-started/> — accessed 2026-06-25

- <https://connectrpc.com/docs/protocol/> — accessed 2026-06-25
- <https://buf.build/blog/connect-a-better-grpc> — accessed 2026-06-25
- <https://github.com/connectrpc/validate-go> — accessed 2026-06-25
- <https://github.com/bufbuild/protovalidate-go/releases> — accessed 2026-06-25
- <https://github.com/sqlc-dev/sqlc/releases> — accessed 2026-06-25 (sqlc v1.31.1)
- <https://docs.sqlc.dev/en/stable/guides/using-go-and-pgx.html> — accessed 2026-06-25
- <https://docs.sqlc.dev/en/latest/reference/datatypes.html> — accessed 2026-06-25
- <https://docs.sqlc.dev/en/latest/howto/overrides.html> — accessed 2026-06-25
- <https://pkg.go.dev/github.com/jackc/pgx/v5> — accessed 2026-06-25 (pgx v5.10.0)
- <https://queryplane.com/blog/postgres-row-level-security-in-practice/> — accessed 2026-06-25
- <https://ricofritzsche.me/mastering-postgresql-row-level-security-rls-for-rock-solid-multi-tenancy/> — accessed 2026-06-25
- <https://aws.amazon.com/blogs/database/multi-tenant-data-isolation-with-postgresql-row-level-security/> — accessed 2026-06-25
- <https://www.bytebase.com/reference/postgres/how-to/postgres-row-level-security/> — accessed 2026-06-25
- <https://redis.io/tutorials/redis-backed-job-queue-for-background-workers/> — accessed 2026-06-25
- <https://redis.io/docs/latest/commands/xclaim/> — accessed 2026-06-25
- <https://oneuptime.com/blog/post/2026-01-21-redis-dead-letter-queue/view> — accessed 2026-06-25
- <https://oneuptime.com/blog/post/2026-03-31-redis-handle-consumer-failures-streams/view> — accessed 2026-06-25
- <https://redis.io/blog/single-shot-reliable-consumers-with-xreadgroup-claim-in-redis-84/> — accessed 2026-06-25
- <https://redis.io/docs/latest/develop/use-cases/streaming/> — accessed 2026-06-25
- <https://github.com/pressly/goose/releases> — accessed 2026-06-25 (goose v3.27.1)
- <https://volomn.com/blog/database-migration-using-atlas-and-goose> — accessed 2026-06-25
- <https://atlasgo.io/blog/2022/12/01/picking-database-migration-tool> — accessed 2026-06-25
- <https://codelit.io/blog/database-migration-tools-comparison> — accessed 2026-06-25
- <https://atlasgo.io/guides/migration-tools/goose-import> — accessed 2026-06-25